

Mainline Testing Document

Mainline testing is to ensure that automated test cases are system-agnostic. Any test case should run on any Android phones, and on both Windows/macOS systems. Make sure to do mainline testing on a pristine code system every time.

WINDOWS

Note: you can only test on Android phones when using a Windows Machine.

Refer to the [CG Windows Framework setup document](#)

Cloning fresh code base: **git clone --branch CardioGuide2.0-Web-API_Integrations**

<https://bitbucket.sjm.com/scm/myc/cardioguideapp-testautomation.git>

```
C:\TC_Verification>git clone --branch CardioGuide2.0-Web-API_Integrations https://bitbucket.sjm.com/scm/myc/cardioguideapp-testautomation.git
Cloning into 'cardioguideapp-testautomation'...
remote: Enumerating objects: 57343, done.
remote: Counting objects: 100% (57343/57343), done.
remote: Compressing objects: 100% (24281/24281), done.
remote: Total 57343 (delta 32398), reused 46755 (delta 22157)
Receiving objects: 100% (57343/57343), 2.12 GiB | 861.00 KiB/s, done.
Resolving deltas: 100% (32398/32398), done.
Updating files: 100% (1369/1369), done.
C:\TC_Verification>
```

Opened IntelliJ and go to cardioguide-testautomation folder

Ran **mvn clean install -DskipTests** to download all libraries. It should say “Build Success”

```
Terminal Local x + v
[INFO] --- surefire:3.1.2:test (default-test) @ cardioguide-testautomation-2.0 ---
[INFO] Tests are skipped.
[INFO] --- jar:3.4.1:jar (default-jar) @ cardioguide-testautomation-2.0 ---
[INFO] Building jar: C:\TC_Verification\cardioguideapp-testautomation\target\cardioguide-testautomation-2.0-1.0-SNAPSHOT.jar
[INFO] --- install:3.1.2:install (default-install) @ cardioguide-testautomation-2.0 ---
[INFO] Installing C:\TC_Verification\cardioguideapp-testautomation\pom.xml to C:\Users\BODANSX\.m2\repository\com\example\cardioguide-testautomation-2.0\1.0-SNAPSHOT\cardioguide-testautomation-2.0-1.0-SNAPSHOT.pom
[INFO] Installing C:\TC_Verification\cardioguideapp-testautomation\target\cardioguide-testautomation-2.0-1.0-SNAPSHOT.jar to C:\Users\BODANSX\.m2\repository\com\example\cardioguide-testautomation-2.0\1.0-SNAPSHOT\cardioguide-testautomation-2.0-1.0-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 30.141 s
[INFO] Finished at: 2026-06-16T15:07:01-04:00
[INFO] -----
PS C:\TC_Verification\cardioguideapp-testautomation>
```

Connect your Android phone and run **adb devices** to find the uiud of your phone. If it says “unauthorized,” make sure you change USB preferences to allow File transfer and turn on USB debugging on the phone.

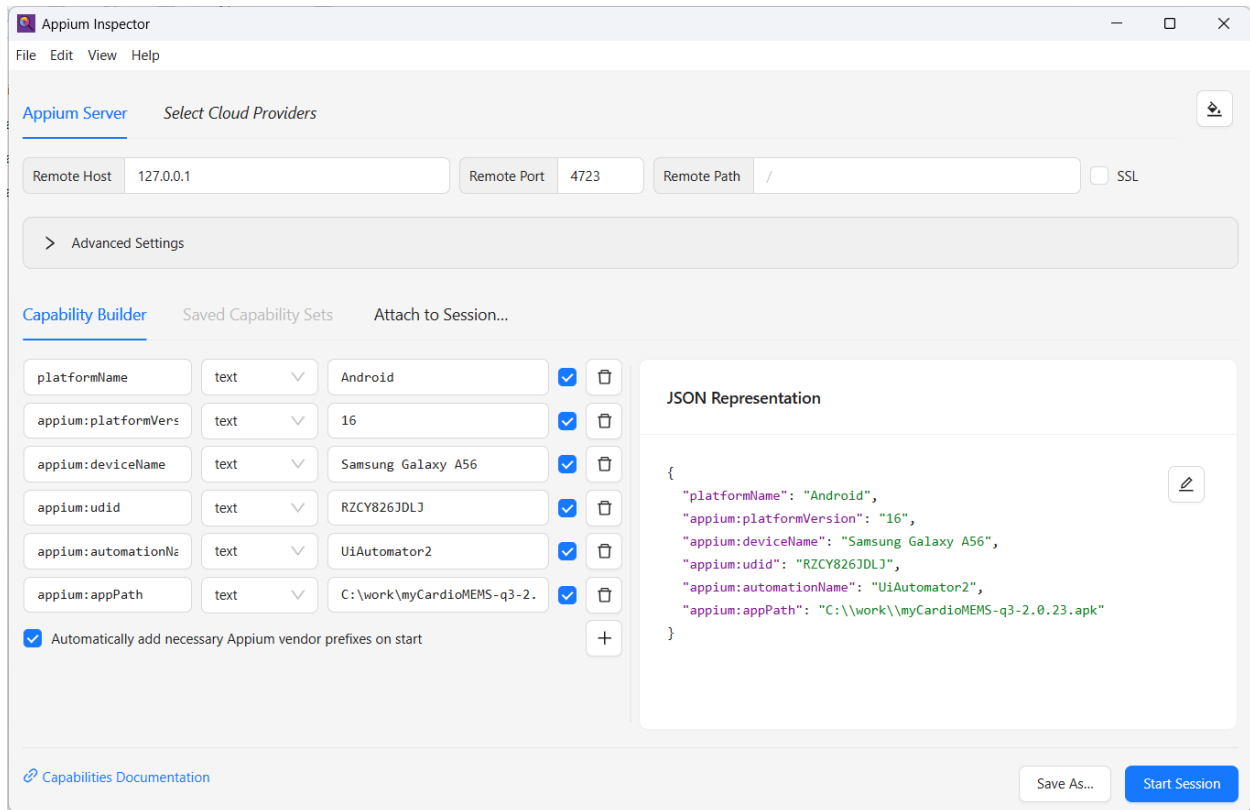
```
Command Prompt
Microsoft Windows [Version 10.0.26100.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\BODANSX>adb devices
List of devices attached
RZCY826JDLJ    device

C:\Users\BODANSX>
```

Paste JSON in Appium Inspector. Make sure all the values are accurate, including latest build apk path. Appium Inspector is useful for looking at xpaths on the device but is not needed to simply run test cases.

```
{
  "platformName": "Android",
  "appium:platformVersion": "16",
  "appium:deviceName": "Samsung Galaxy A56",
  "appium:udid": "RZCY826JDLJ",
  "appium:automationName": "UiAutomator2",
  "appium:appPath": "C:\\work\\ myCardioMEMS-q3-2.0.23.apk "
}
```



Most test cases are on a separate branch. If you have just cloned the repo, you can directly switch into your testing branch. Ex **git switch notification19_2**. Or else, run the following commands to update your branch:

If you are currently on CardioGuide2.0-Web-API_Integrations branch, do

Git pull

Git switch notification19_2

Git pull

If you are on a branch and want to switch to another testing branch, do

Git stash

Git switch CardioGuide2.0-Web-API_Integrations

Git pull

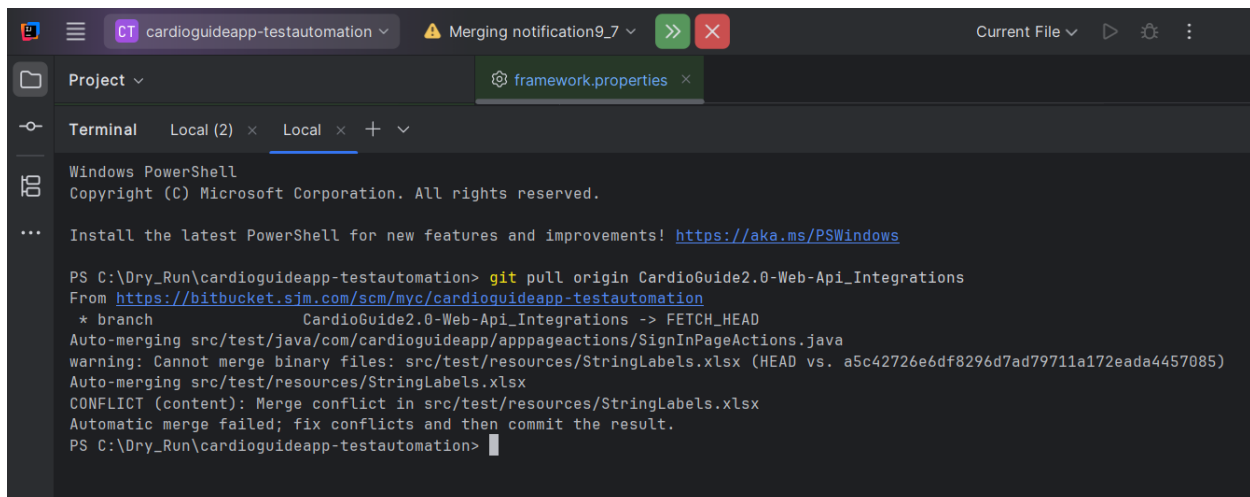
Git switch notification19_2

Git pull

If you are testing an older PR, you may need to do **git pull origin CardioGuide2.0-Web-API_integrations** after doing git pull in your testing branch and resolve any merge conflicts:

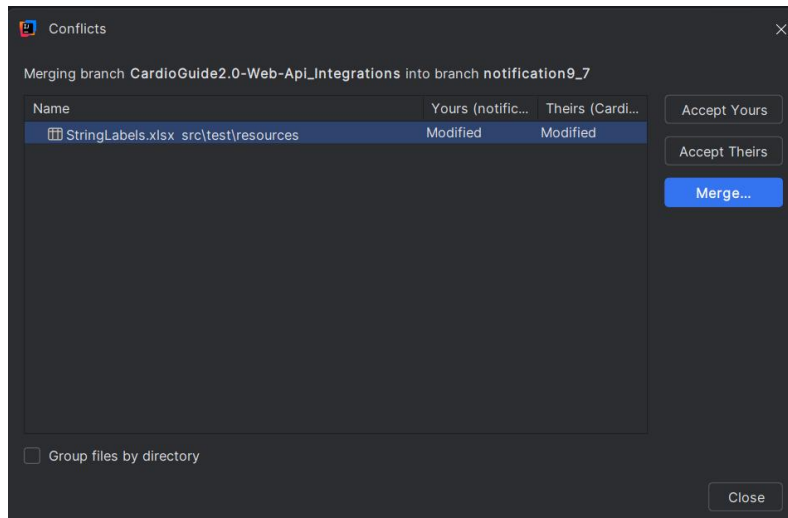
Resolving Merge conflicts

After running git pull origin CardioGuide2.0-Web-API_Integrations, if there are merge conflicts, a green and red button will appear at the top of the screen next to the branch name.



```
PS C:\Dry_Run\cardioguideapp-testautomation> git pull origin CardioGuide2.0-Web-API_Integrations
From https://bitbucket.sjm.com/scm/myc/cardioguideapp-testautomation
 * branch          CardioGuide2.0-Web-API_Integrations -> FETCH_HEAD
Auto-merging src/test/java/com/cardioguideapp/apppageactions/SignInPageActions.java
warning: Cannot merge binary files: src/test/resources/StringLabels.xlsx (HEAD vs. a5c42726e6df8296d7ad79711a172eada4457085)
Auto-merging src/test/resources/StringLabels.xlsx
CONFLICT (content): Merge conflict in src/test/resources/StringLabels.xlsx
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Dry_Run\cardioguideapp-testautomation>
```

Press the green >> button. You will see a popup that shows all the files with all the files that have merge conflicts. You can open the files outside of the popup and see what the conflicts are. If you press “Accept Yours,” you will be accepting the version of the file on the branch you are currently on. If you press “Accept Theirs,” you will be accepting the CardioGuide2.0-Web-API_Integrations version of the file.



After fixing the merge conflicts, run **git commit** to finish the merge. You will be brought to an editable file. Exit the file by typing **:qa** and pressing enter. Verify there is no green or red button next to the name of the branch.

In `src\test\resources\suites\testng.xml`, edit the class tag to match the path of the test case you are running. If the PR you are testing has multiple test cases, you can add more class tags and the automation will run in order. You may have to configure Maven and the JDK in order for errors in `sanity.xml` to resolve.

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >
<suite name="CardioGuide Automation Suite" verbose="1" parallel="false">
  <listeners>
    <listener class-name="com.cardioguide.framework.reports.SoftFailureFlushListener"/>
  </listeners>
  <test name="Requirements Based TestCase">
    <groups>
      <run>
        <!-- Comment group name line below if you don't want group name to apply-->
        <!-- <include name="readings"/>-->
      </run>
    </groups>
    <classes>
      <class name="com.cardioguideapp.tests.journal.DRXTMAPP252More_Weight24Test"/>
      <class name="com.cardioguideapp.tests.journal.DRXTMAPP251More_Weight25USTest"/>
    </classes>
  </test>
</suite>
```

Or, you can add a tag in your command **-Dtest=DRXTMAPP4072Notification19_2** to run specific tests. Ex: `Mvn clean test "-DplatformName=Android" "-Ddevice.udid=e2ea92f8" "-Dapp=C:/work/27.apk" "-Dtest=DRXTMAPP16SignIn9Test, DRXTMAPP16SignIn9Test"`

Start appium in IDE terminal (**appium**)

In a separate terminal, run the maven command to run the tests:

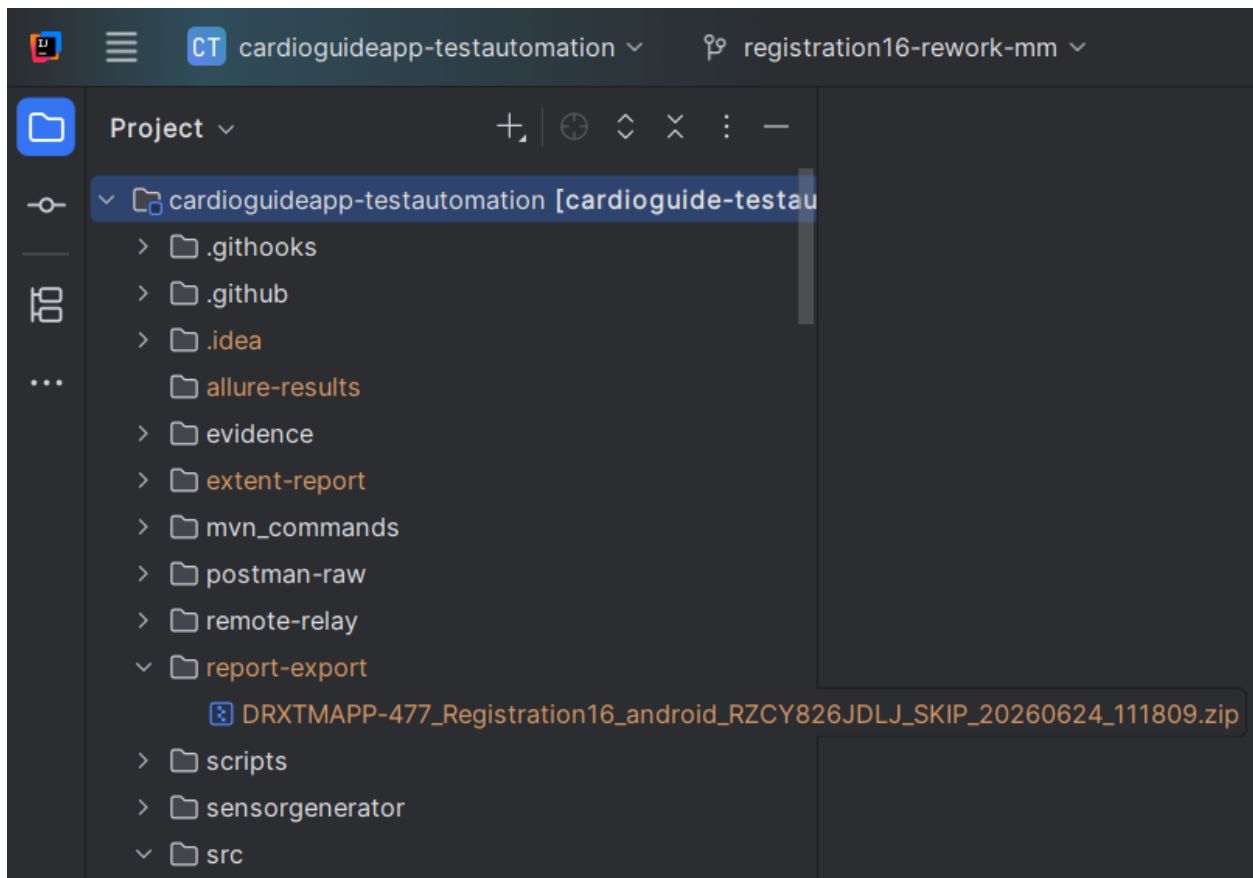
Running on Windows with Android: **`mvn clean test "-DplatformName=Android" "-Ddevice.udid=RZCY826JDLJ" "-Dapp=C:/work/23.apk"`**

THE ONLY EDITS YOU SHOULD MAKE:

1. Change execution.machine in framework.properties to match computer OS
2. Change common users in framework.properties attached to your phone number
3. Change qa.platform to android/ios based on the phone you are testing
4. Change patient.phone in framework.properties
5. If [Springboot and automatic OTP](#) is configured, Change the SMS forwarder IP address. IP address will change based on different WiFi connections.
6. Some test cases use an already enrolled patient to log in. You can change the patient information in either framework.properties COMMON USER or change it in the test case json file.

You can keep a local copy of framework.properties and use that every time you run a test.

Download the allure report from report-export folder. If you have multiple reruns match the time stamps of the zip folders with your latest time. The format is YYYY-MM-DD_HHMMSS.zip. Right click your folder and press Open In > Explorer. You can copy and paste this zip folder to share with others.



Add a comment in the PR request and in the Automation ticket. Make sure to include the operating system, the report zip folder and whether the test case failed or not.

Ex. Comment in PR request:

@PR Owner

Test <Manual Test Case Name> was executed on pristine Windows setup using Samsung Galaxy A56.

Allure report updated in: <link to comment in automation ticket with allure report>

Test executed successfully.

MACBOOK

You can run test cases on both Android and iOS using a Mac.

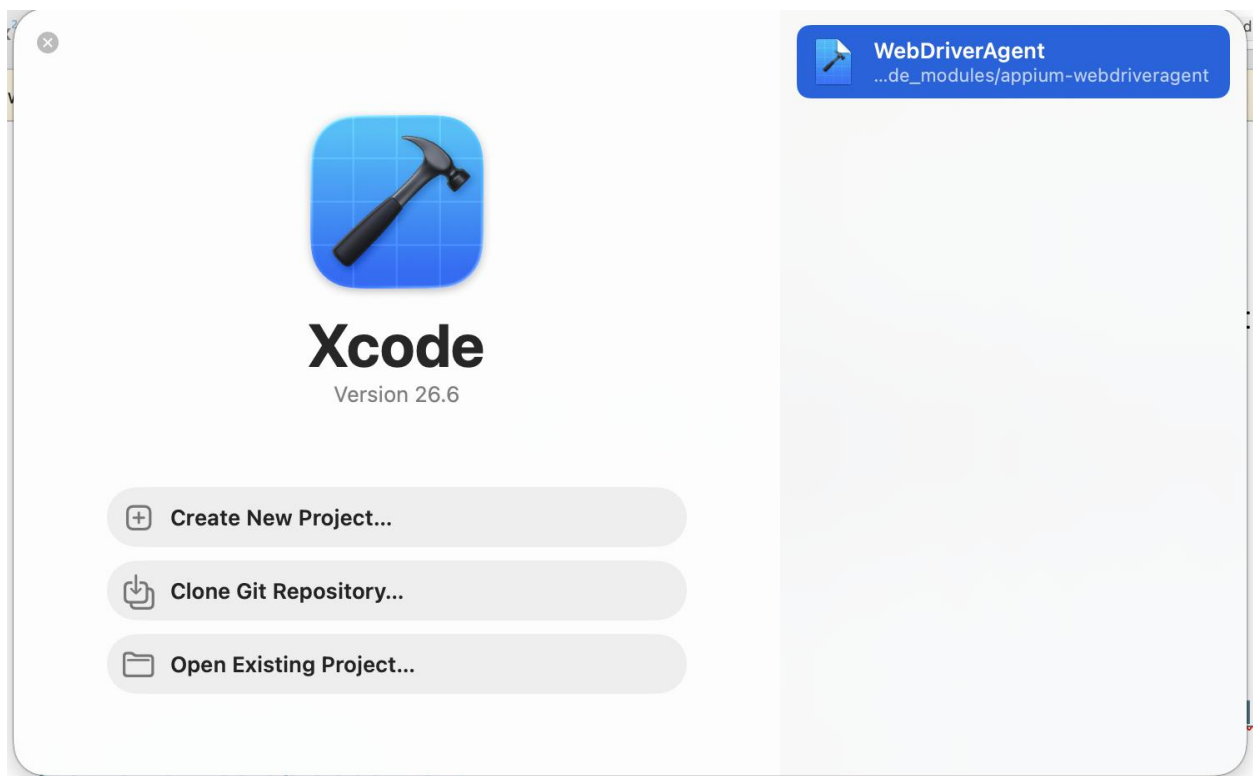
Running automation on Mac for Android is similar to Windows with Android. Use this command to run test cases on Android with a Mac:

Running on Mac with Android: **mvn clean test -DplatformName=Android -Ddevice.udid=RFCY90ZGW6A -Dapp=/Users/BODANSX/Downloads/26.apk**

Running Automation on iOS:

Connect your phone to the computer and the phone has Developer Mode turned on. Turn it on by going to iPhone settings > Privacy & Security > Developer Mode and toggle it ON

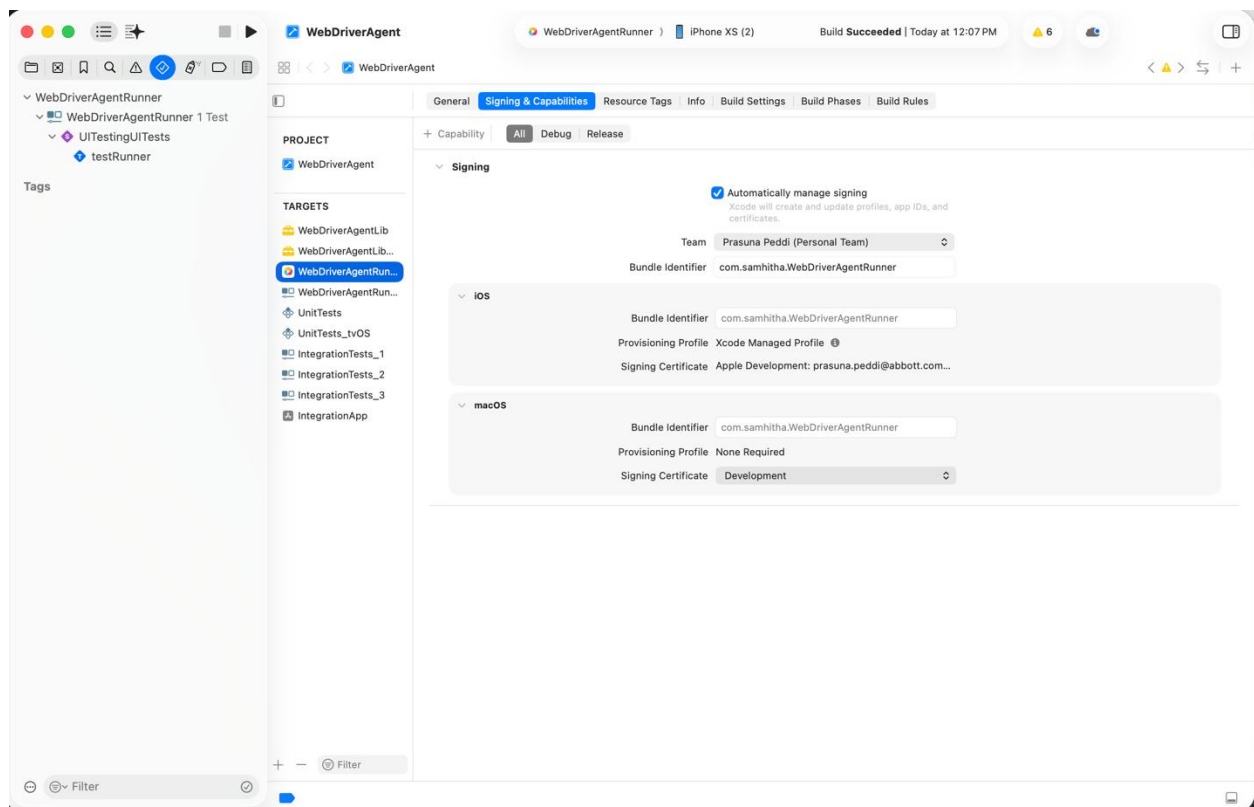
Once Xcode is downloaded open the WebDriverAgent project. Refer to the [CG Macbook Framework setup document](#)



Open the WebDriverAgentRunner. Tap on Signing & Capabilities. At the top of the screen, make sure it says WebDriverAgentRunner > iPhone model.

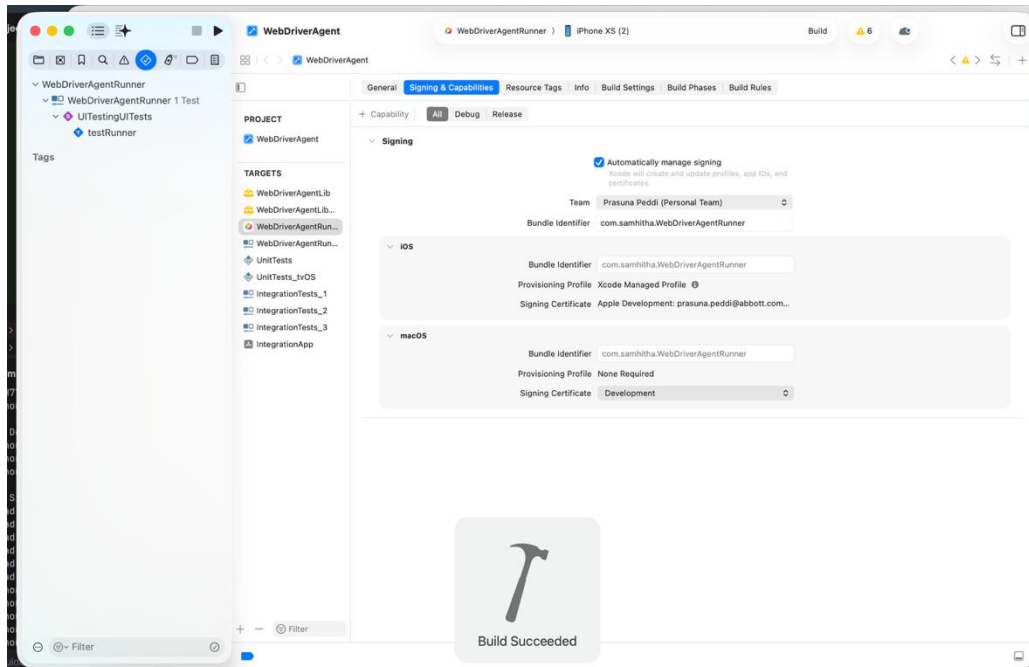
In “Team”, sign in with your personal or Abbott apple ID.

Note: If you sign in with your personal Apple ID, you have a maximum of 2-3 iPhones you can register with your apple id. This means you cannot test on multiple different iPhones. If you register the maximum number of phones, sign in with another apple id in the team section to test more phones



Press Product > Test or run **Cmd + B** to start the build. You should see a popup that says “Build Succeeded.” If you have any errors, change the Bundle Identifier in the Signing and iOS sections. By default, the Bundle identifiers will have “com.facebook.WebDriverAgentRunner.” Change the facebook word to a unique work and

retry the build. You may need to change the Bundle Identifiers in the WebDriverAgentRunner Section and the WebDriverLib section.



If you have other errors, at the top of the screen, there will show a small red x and a number next to it. If you tap it, it will show you each error on the left side of the screen. You may need to trust the developer Apple id on the phone. Go to iPhone settings > General > VPN & Device Management. There should be a section that says Developer App and should list the email of the apple id used in Xcode. Tap the email and select Allow.

Paste this JSON for iOS in Appium Inspector.

```
{  
  
  "platformName": "iOS",  
  
  "appium:platformVersion": "18.5",  
  
  "appium:deviceName": "iPhone SE",  
  
  "appium:automationName": "XCUITest",  
  
  "appium:udid": "00008110-001E5CD11EEA201E"
```

```
}
```

After the Xcode build has succeeded, you can run automation test cases in IntelliJ. Run **appium** in a separate terminal. Use **xcrun xctrace list devices** to get the uuid value for your iPhone. The uuid of the phone is the value underneath the name of the Macbook.

```
[xc%  
BODANSX@M9H7Y7QC4Y ~ % xcrun xctrace list devices  
== Devices ==  
M9H7Y7QC4Y (63EC88E0-92A3-58B2-80AB-E387A864DE8E)  
iPhone XS (2) (18.6.2) (00008030-0019794E3AE1802E)  
  
== Devices Offline ==  
iPhone XS (18.7.7) (00008020-00084DDE3A99002E)  
iPhone16e (18.6.2) (00008140-001C02E114DB801C)  
iphonese (18.5) (00008110-001E5CD11EEA201E)  
  
== Simulators ==  
iPad (A16) Simulator (26.5) (1E37D5AB-7E38-45BA-B67C-59CB15232112)  
iPad Air 11-inch (M4) Simulator (26.5) (FEFAB440-40E3-47EE-A7F4-16765B4C3991)  
iPad Air 13-inch (M4) Simulator (26.5) (1590CC98-E049-4D6B-BCE9-49636D57B9AA)  
iPad Pro 11-inch (M5) Simulator (26.5) (FBA253E7-434B-49CF-9093-98DFBF1F3E64)
```

Run the below command in a separate IntelliJ terminal to start the test case. Make sure you change the **qa.platform** to **ios** before running the script

Running on Mac with iPhone: **mvn clean test -DplatformName=iOS -
Ddevice.udid=00008030-0019794E3AE1802E**